Car Rental Fleet Management

Vadlapatla Jaswin 2620030148

Viswa Aditya 2620040108

Abstract

- This project is a simple Car Rental Fleet Manager program that helps a rental business keep track of its cars, bookings, and returns without the usual paperwork hassle. It lets the admin add new cars to the fleet, rent them out to customers, process returns, and keep an eye on which vehicles need maintenance.

  To make all this work smoothly, the program relies on some basic but powerful programming logic. Whenever someone wants to rent a car, an if-else check makes sure that car is actually free before confirming the booking, so two people can't accidentally book the same vehicle. A switch statement is used to sort cars by type—like hatchback, sedan, or SUV—so each one can be rented at the right price. To help the admin see the entire fleet at a glance, a for loop goes through the list of cars and displays them all, showing what's available and what's already rented out. And to keep the whole system running smoothly, a while loop keeps the program active in a loop, letting the user rent, return, or check on cars again and again until they decide to exit.

Understanding Problem Statement

- **Manual tracking is error-prone:** keeping track of cars, bookings and returns on paper is slow and leads to mistakes.
- **Double booking risk:** two customers can accidentally book the same vehicle if availability is not checked.
- **Charges vary:** the total depends on days rented, the rate for each vehicle type, and late fees.
- **Maintenance gaps:** cars that need service must be flagged and kept out of circulation until cleared.
- **Goal:** a Java program that handles the fleet, rentals, returns and maintenance using control statements.

Module 1: From Problem to Program

- **Problem solving:** computational thinking, algorithm and flowchart
- **Java basics:** Java environment, program structure, primitive data types, variables, constants, literals
- **Operators:** arithmetic, relational, logical, assignment, ternary, unary, special, bitwise
- **Expressions:** typecasting, precedence and associativity
- **Console I/O:** formatted printing, reading input with Scanner

Module 2: Control Flow

- **Selection:** if, if-else, nested-if, if-else-if ladder
- **Switch:** default, break, compound statements
- **Loops:** while, do-while, for
- **Loop control:** continue, infinite loops, nested loops

Module 3: Methods & Arrays

- **Methods:** definition, invocation, parameter passing, return values, overloading
- **Recursion:** base case and recursive case
- **Loops:** enhanced for-each loop
- **1D arrays:** traversal, searching, summation, average, counting
- **2D arrays:** matrix arithmetic

Module 4: OOP & Exceptions

- **Classes and objects:** fields, methods, new, null, constructors (default, parameterized, overloading), this
- **Encapsulation:** private fields, accessors/mutators, static members, toString()
- **Inheritance:** is-a relationships, method overriding, interfaces, programming to interfaces
- **Exceptions:** checked vs unchecked, try-catch-finally, multi-catch, throw, throws

Module 5: Strings & File I/O

- **Strings:** immutability, length, substring, comparison (equals vs ==), StringBuilder and StringBuffer
- **Text handling:** split(), join(), trimming, validation, tokenization
- **Library algorithms:** Arrays.sort(), Collections.sort(), binarySearch(), Comparable and Comparator
- **File I/O:** Files and Path, buffered readers/writers, try-with-resources

Module 6: Collections & Streams

- **Collections:** List, Set, Map using ArrayList, HashMap, HashSet; generics and autoboxing
- **Functional style:** lambda expressions, method references, Predicate, Function, Consumer, Supplier
- **Streams:** filter, map, sorted, distinct, limit; forEach, collect, count, reduce
- **Collectors:** toList, groupingBy, counting, joining

Conclusion

- **Complete solution:** the Car Rental Fleet Manager covers 4 features: add vehicles, rent car, return car and maintenance tracking.
- **4 control statements:** if-else checks availability, switch handles vehicle type, for loop displays the fleet, while loop runs the rental menu.
- **Takeaway:** a few simple programming tools, used the right way, can solve a real everyday business problem.

Screen Shots

Main.java

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;
import java.util.stream.Collectors;

// Custom Exception for Rental Errors (Module 4)
class CarNotAvailableException extends Exception {
    public CarNotAvailableException(String message) {
        super(message);
    }
}

// Interface representing rent behavior (Module 4)
interface Rentable {
    void rentCar() throws CarNotAvailableException;
    void returnCar();
}

// Car Class implementing Encapsulation and Interface (Module 4)
class Car implements Rentable {
    private String carId;
    private String brand;
    private String model;
    private double pricePerDay; // in INR
    private boolean isAvailable;

    // Parameterized Constructor
    public Car(String carId, String brand, String model, double pricePerDay) {
        this.carId = carId;
        this.brand = brand;
        this.model = model;
        this.pricePerDay = pricePerDay;
```

Console | I/O | AI Agent — Running

```
   CAR RENTAL MANAGEMENT SYSTEM (UNIFLOW)
========================================
1. View All Cars
2. View Available Cars (Using Streams)
3. Rent a Car
4. Return a Car
5. Exit
Enter your choice (1-5): 1

--- Complete Fleet Inventory ---
ID: C101 | Maruti Swift | Rate: ₹2200.00/day | Status: Available
ID: C102 | Tata Nexon | Rate: ₹2800.00/day | Status: Available
ID: C103 | Mahindra Scorpio-N | Rate: ₹4500.00/day | Status:
Available
ID: C104 | Hyundai Creta | Rate: ₹3200.00/day | Status:
Available

========================================
   CAR RENTAL MANAGEMENT SYSTEM (UNIFLOW)
========================================
1. View All Cars
2. View Available Cars (Using Streams)
3. Rent a Car
4. Return a Car
5. Exit
Enter your choice (1-5):
```

Geo Tag Photo and git hub link